□ Problems Encountered, Analysis, and Solutions

Research Paper — Technical Issues & Resolutions

Date: April 17, 2026

□ 1. APT Package Manager Failure (Hash Sum Mismatch & Missing Files)

One of the earliest and critical issues encountered during system setup was the failure of the Advanced Package Tool (APT) while updating repositories. Errors such as "Hash Sum mismatch" and missing files in /var/lib/apt/lists/partial indicated corruption in the local package cache. This typically occurs due to interrupted downloads, unstable internet connections, or inconsistencies between repository metadata and locally cached files. As a result, essential packages such as Cloudflare could not be installed through standard repository methods.

To resolve this issue, the corrupted cache was completely cleared, and the required directory structure was manually restored. This ensured that APT could re-fetch repository metadata from scratch without relying on inconsistent cached data.

```
sudo killall apt apt-get
sudo rm -rf /var/lib/apt/lists/*
sudo mkdir -p /var/lib/apt/lists/partial
sudo chmod -R 755 /var/lib/apt/lists
sudo apt update
```

This solution effectively reset the package management system, allowing normal installation processes to resume.

□ 2. Cloudflared Package Unavailability in APT Repository

Following the APT issues, an attempt to install Cloudflare Tunnel using apt install cloudflared failed due to the package not being found in the repository. This can occur when repository metadata is outdated, corrupted, or when the package is not included in default Ubuntu repositories.

To overcome this limitation, a manual installation approach was adopted by downloading the .deb package directly from Cloudflare's official GitHub releases. This bypassed the dependency on APT repositories entirely.

```
curl -L
https://github.com/cloudflare/cloudflared/releases/latest/download/cloudfl
ared-linux-amd64.deb -o cloudflared.deb
```

```
sudo dpkg -i cloudflared.deb
sudo apt --fix-broken install -y
```

This approach ensured successful installation while maintaining control over dependencies.

3. QUIC Protocol Failure (Timeout Errors in Tunnel Connection)

During the initial attempt to establish a Cloudflare Tunnel, repeated connection failures were observed with errors indicating "failed to dial a QUIC connection". QUIC is a transport protocol built on UDP, and many institutional or campus networks restrict UDP traffic for security reasons. As a result, the tunnel was unable to establish a connection with Cloudflare's edge servers.

The issue was resolved by explicitly forcing the tunnel to use HTTP/2 instead of QUIC. HTTP/2 operates over TCP, which is universally allowed across networks, ensuring stable connectivity.

```
cloudflared tunnel --url http://localhost:11434 --protocol http2
```

This modification allowed the tunnel to bypass network restrictions and establish a successful connection.

4. DNS Resolution Failure (NXDOMAIN Error)

After successfully generating a public URL, attempts to access the service later resulted in a DNS_PROBE_FINISHED_NXDOMAIN error. This occurred because the tunnel being used was a temporary (quick) tunnel, which exists only while the cloudflared process is actively running. Once the process terminates or the system shuts down, the associated DNS entry is removed.

The solution was to restart the tunnel, which generates a new temporary URL:

```
cloudflared tunnel --url http://localhost:11434 --protocol http2
```

This highlighted the importance of understanding session-based services and their lifecycle.

5. PowerShell Curl Incompatibility (Command Execution Failure)

When attempting to test the API from a Windows system, the curl command failed due to PowerShell interpreting it as an alias for Invoke-WebRequest. Unlike standard Linux curl, PowerShell's implementation does not support the same flags or syntax, leading to errors in request execution.

To resolve this, either the native executable (curl.exe) was used, or PowerShell's own REST method was utilized.

```
curl.exe -X POST "URL" -H "Content-Type: application/json" -d
"{\"model\":\"llama3\",\"prompt\":\"hello\",\"stream\":false}"
```

OR

```
Invoke-RestMethod -Uri "URL" -Method POST -ContentType "application/json"
-Body $body
```

This ensured compatibility across different operating systems.

6. Command Splitting Issue in PowerShell

Another issue encountered was the improper execution of multi-line commands in PowerShell. When commands were pasted with line breaks, PowerShell executed each line separately, causing parameters such as -H and -d to be interpreted as independent commands.

This was resolved by ensuring that commands were written in a single line or by using PowerShell's line continuation character (backtick `).

```
curl.exe -X POST "URL" -H "Content-Type: application/json" -d
"{\"model\":\"llama3\",\"prompt\":\"hello\",\"stream\":false}"
```

This ensured proper parsing and execution of the command.

7. 403 Forbidden Error (Cloudflare Blocking Requests)

A major issue occurred when API requests returned a 403 Forbidden response. This was caused by Cloudflare's security mechanisms identifying the request as non-browser or automated traffic. Requests lacking typical browser headers were flagged and blocked.

The solution involved adding headers such as User-Agent and Accept to mimic a legitimate browser request.

```
Invoke-RestMethod -Uri "URL" -Method POST -ContentType "application/json"
`
-Headers @{ "User-Agent"="Mozilla/5.0"; "Accept"="*/*" } `
-Body $body
```

This allowed the request to pass through Cloudflare's filtering mechanisms successfully.

8. API Endpoint Misunderstanding (Blank or Error Page in Browser)

Initially, accessing the tunnel URL in a browser resulted in a blank page or error, leading to confusion. This occurred because Ollama exposes a REST API rather than a web interface, and the root endpoint (/) does not return meaningful content.

The correct approach was to use specific API endpoints such as /api/generate or /api/tags.

```
curl http://localhost:11434/api/generate -d '{
  "model":"llama3",
  "prompt":"hello",
  "stream":false
}'
```

This clarified the difference between API servers and web servers.

9. Dependency on Running Services (Tunnel and Ollama)

The system required both the Ollama server and the Cloudflare tunnel to be running simultaneously. If either service stopped, the entire system became inaccessible. This dependency highlighted the importance of process management in distributed systems.

To ensure proper operation, both services were started and monitored:

```
ollama serve
cloudflared tunnel --url http://localhost:11434 --protocol http2
```

10. Network Environment Restrictions and Variability

The project demonstrated that network behavior varies significantly depending on the environment. While certain configurations worked in home networks, they failed in institutional networks due to stricter firewall rules and protocol restrictions.

This required adaptive troubleshooting, including protocol switching, manual installations, and testing across different networks. It emphasized the importance of designing systems that can operate under varying network constraints.